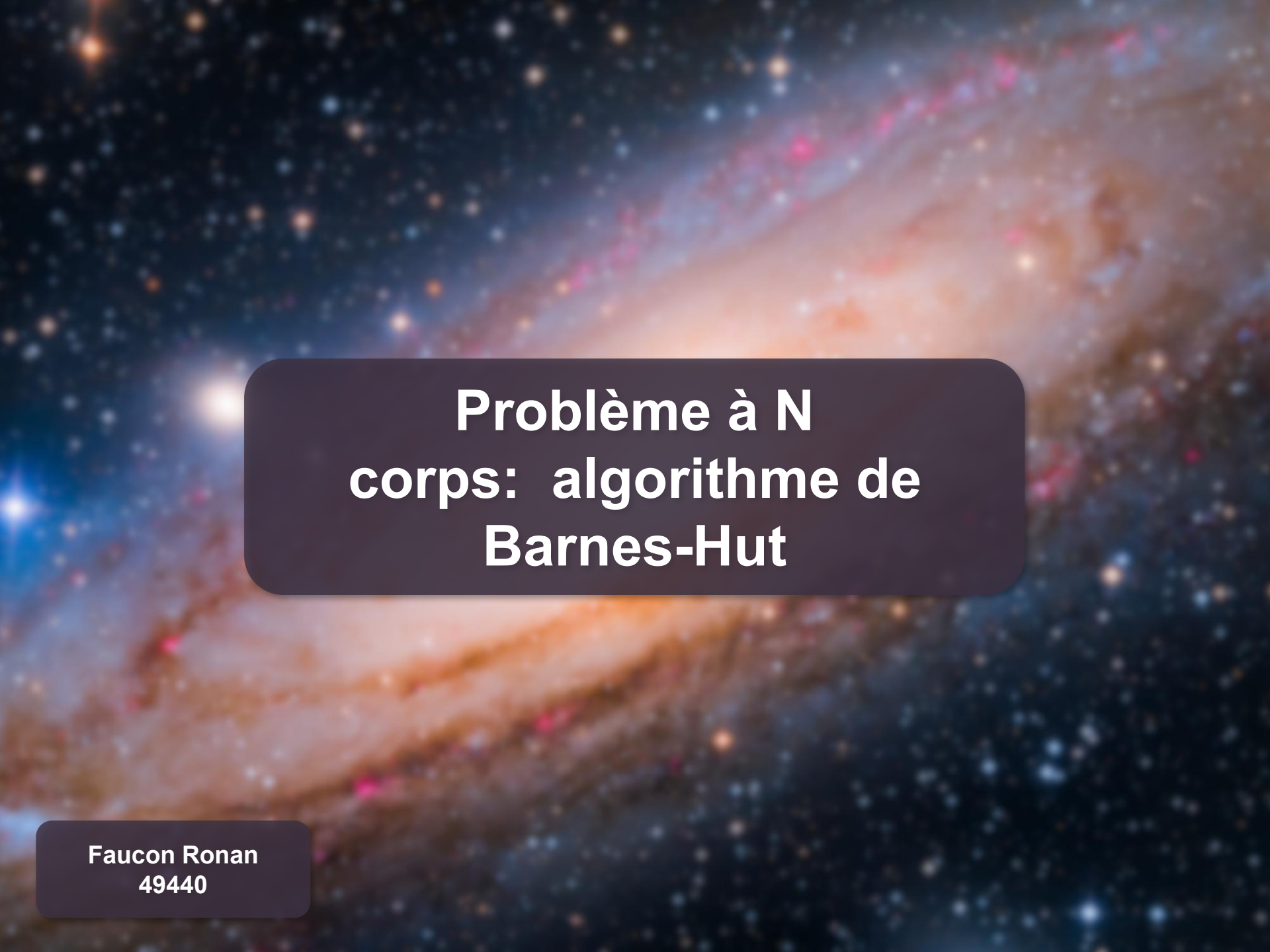


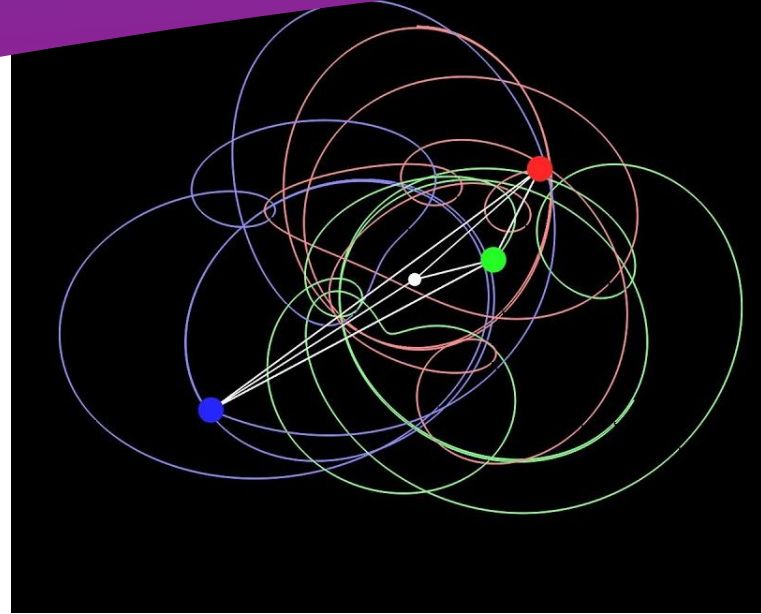


# **Problème à N corps: algorithme de Barnes-Hut**

**Faucon Ronan  
49440**

# Positionnement du problème

Problème à N-corps



Voie lactée:

- $1.5 \times 10^{11}$  d'étoiles
- $2.25 \times 10^{22}$  calculs de forces !!!!

On a besoin d'une meilleure approche:  
l'algorithme de Barnes-Hut

# Objectifs

1. Implémenter l'algorithme Naïf et calculer sa complexité

# Objectifs

1. Implémenter l'algorithme Naïf et calculer sa complexité
2. Découvrir l'algorithme de Barnes-Hut

# Objectifs

1. Implémenter l'algorithme Naïf et calculer sa complexité
2. Découvrir l'algorithme de Barnes-Hut
3. L'implémenter et calculer sa complexité

# Objectifs

1. Implémenter l'algorithme Naïf et calculer sa complexité
2. Découvrir l'algorithme de Barnes-Hut
3. L'implémenter et calculer sa complexité
4. Vérifier et comparer les complexités de ces algorithmes

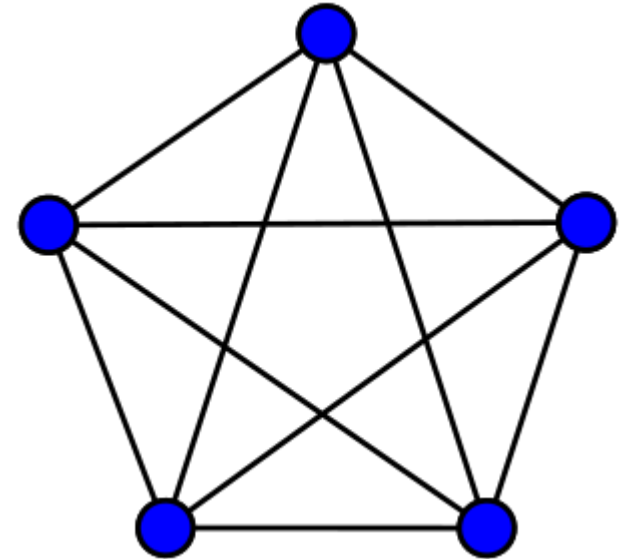
# Objectifs

1. Implémenter l'algorithme Naïf et calculer sa complexité
2. Découvrir l'algorithme de Barnes-Hut
3. L'implémenter et calculer sa complexité
4. Vérifier et comparer les complexités de ces algorithmes
5. Observer des pistes d'amélioration

# Algorithme Naïf

## Pseudo code

```
def Naïf:  
  pour chaque e dans liste_etoile:  
    pour chaque f dans liste_etoile\{e}  
      force ← f.champ(e)  
      e.appliquer(force)
```



## Complexité:

Soit une galaxie à  $N$  étoiles

Pour chacune des  $N$  étoiles il y a  $N-1$  forces à calculer

Donc asymptotiquement cette fonction est en  $O(N^2)$

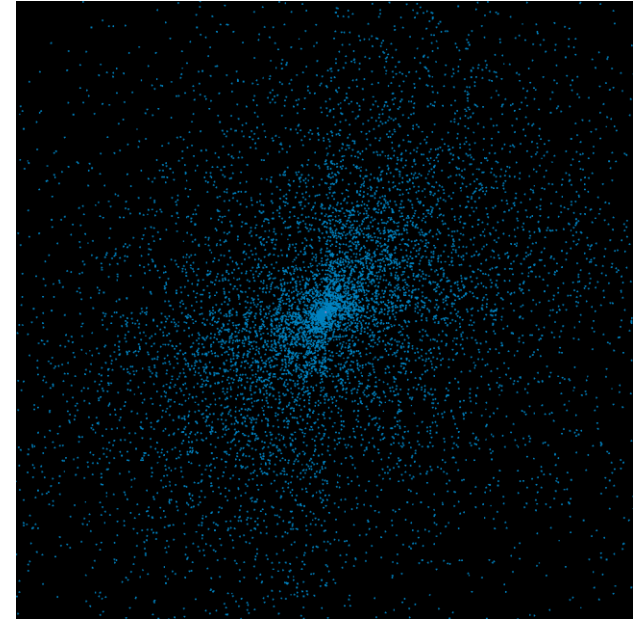


# Algorithme Naïf: Implémentation

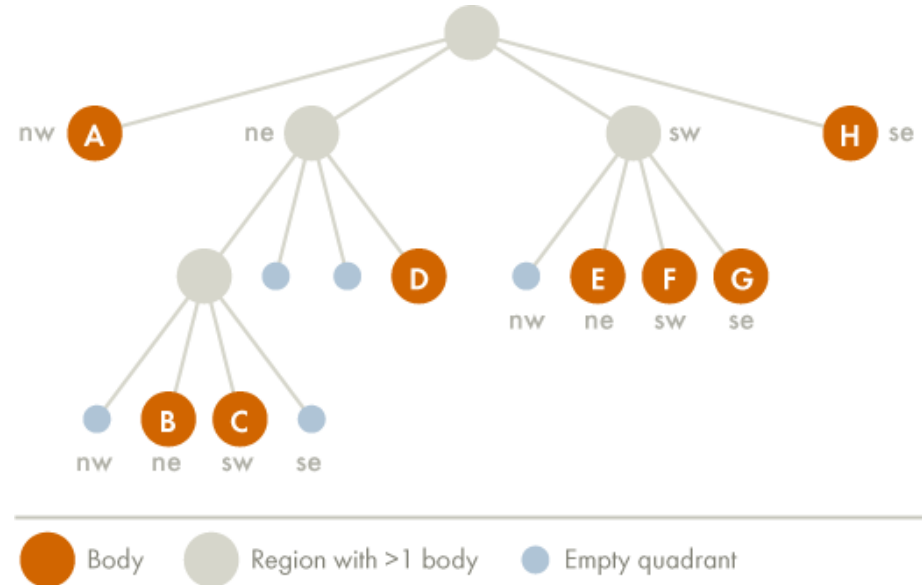
```
class Star{
    Vector2 speed;      // [m s-1]
    Vector2 position;   // [AU]
    double mass;        // [M_SOL]
    const double max_speed = 10;
};
```

```
void Galaxy::Update(){
    for(int i = 0; i < this->star_count; i++)
    {
        this->Naive(i);
        this->stars[i].update_position(this->time_step);
    }
}
```

```
void Galaxy::Naive(int star_id){
    for(int i = 0; i < this->star_count; i++){
        if(i == star_id)
            i++;
        Vector2 F = this->stars[i].force_field(this->stars[star_id].position - this->stars[i].position);
        this->stars[star_id].apply_force(F, this->time_step);
    }
}
```

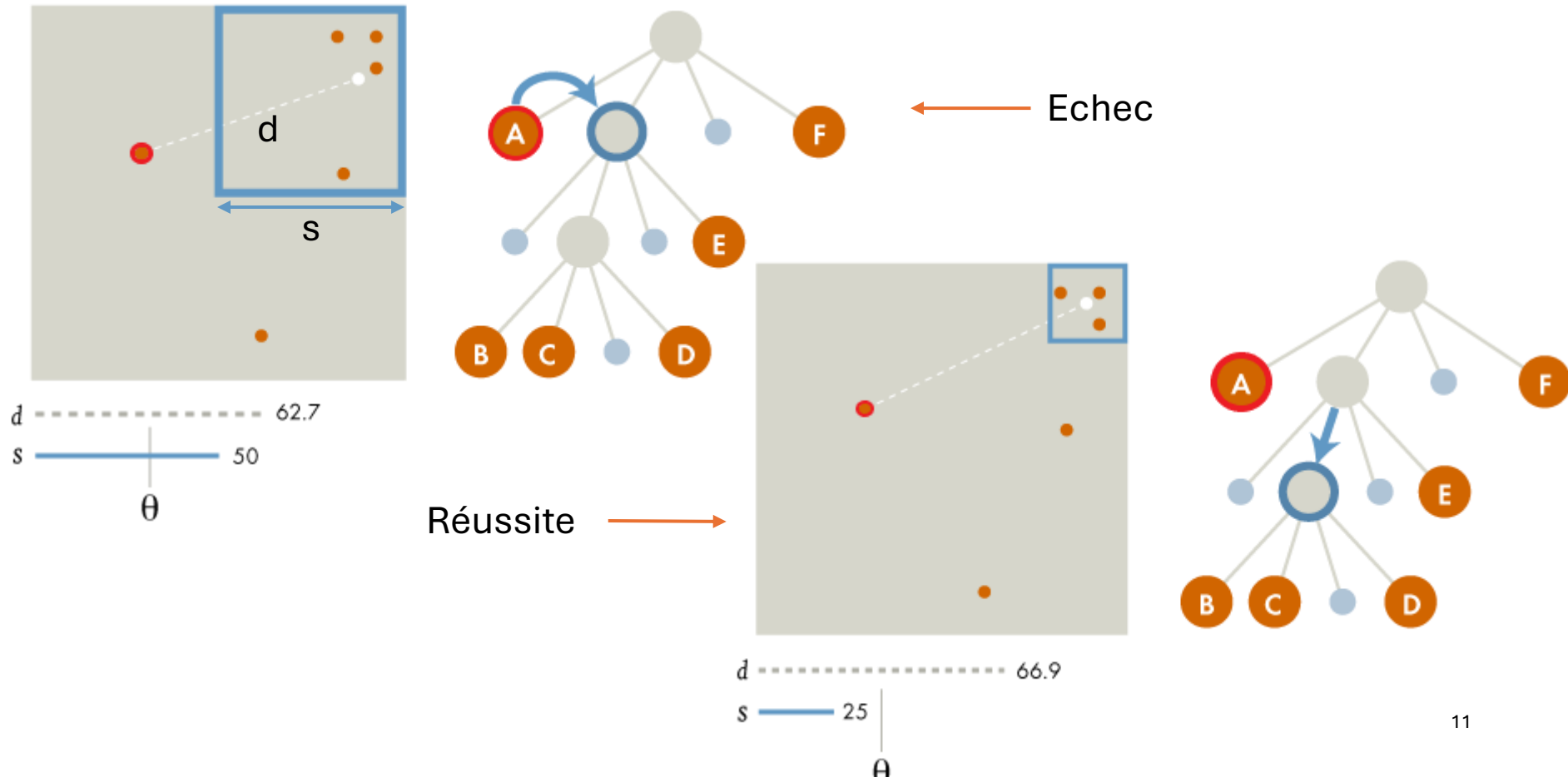


# Algorithme de Barnes-Hut



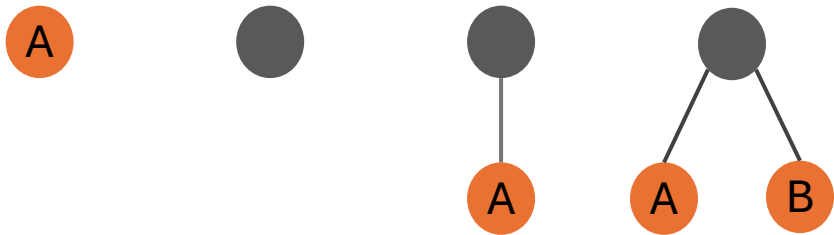
# Algorithme de Barnes-Hut

Critère:  $s/d \leq \theta$



# Barnes-Hut: Implémentation

## Construction de l'arbre



Adaptation

```
class QuadTree{
    QuadTree *childs[4];
    //      ^
    //      |
    //  0   |   1
    //  ---+--->
    //  2   |   3
    //
    std::vector<int> star_list;
    Vector2 mass_center; // [AL]
    Vector2 quad_center; // [AL]
    double mass;         // [M_SOL]
    int depth;
};
```

```
def inserer_dans_arbre(étoile, arbre):
    si arbre ne contient pas d'étoile:
        trouver quadrant de l'étoile
        si il existe une étoile dans le quadrant:
            mettre à jour masse et centre de masse
            inserer_dans_arbre(étoile, arbre[quadrant])
        si le quadrant est vide:
            mettre l'étoile dans cette feuille
    sinon:
        si l'arbre contient plusieurs étoiles ou sa largeur est inférieur à EPSILON:
            mettre à jour centre de masse et masse de l'arbre
            ajouter l'étoile à l'arbre
        sinon:
            mettre à jour la masse et le centre de masse de l'arbre
            récupérer des données de l'étoile déjà présente et l'enlever de l'arbre
            insérer à nouveau les deux étoiles dans cet arbre
```

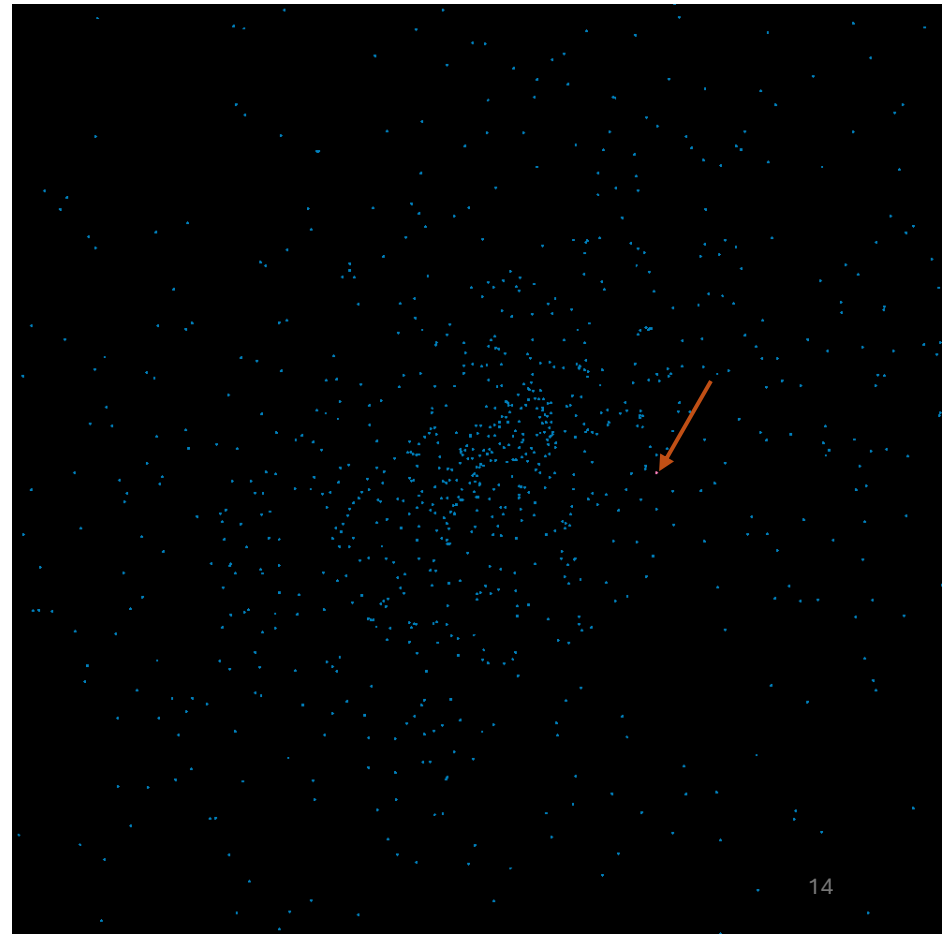
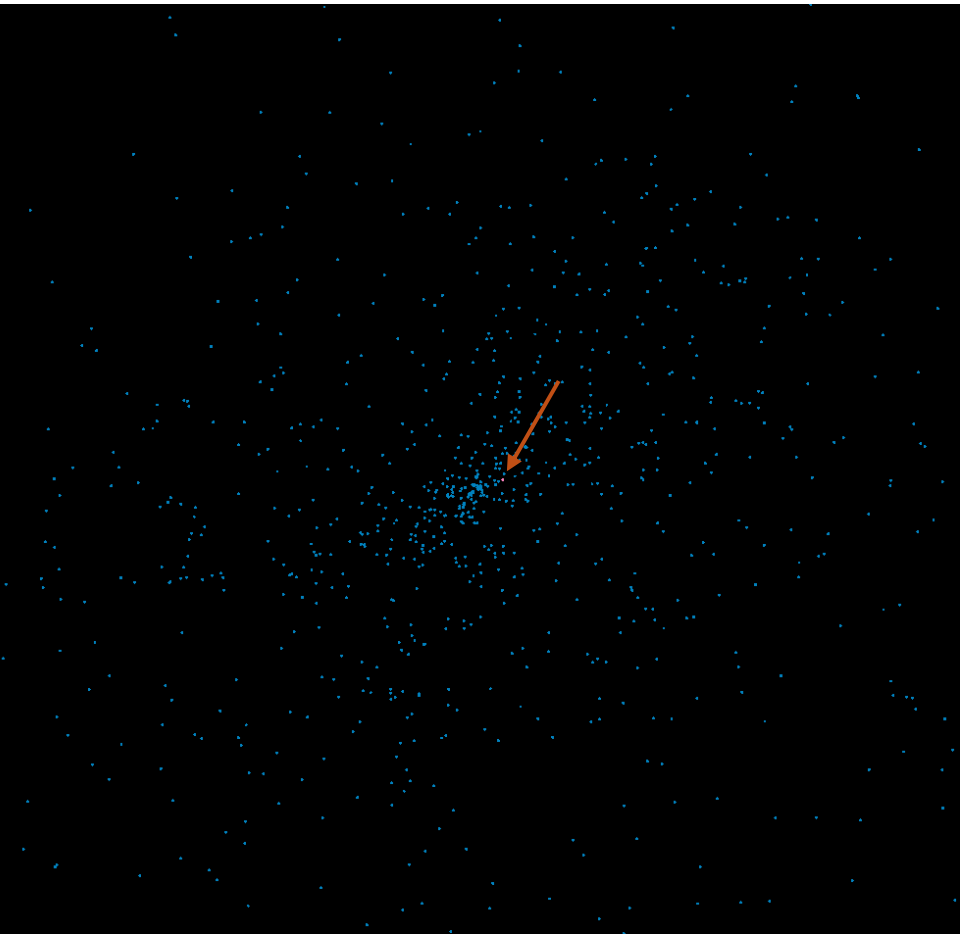
# Barnes-Hut: Implémentation

## Calcul des forces

```
def Calculer_BH_force(étoile, arbre):  
    Queue ← créer une queue d'arbres  
    ajouter à Queue les quadrants non vides  
  
    tant que Queue est non vide:  
        quad = Queue.pop()  
        si quad contient 1 étoile:  
            calculer et appliquer la force  
        sinon si quad contient étoile:  
            Calculer les forces de toutes les étoiles dans quad différentes de étoile  
        sinon si quad contient plusieurs étoiles:  
            On calcule la force créée par le centre de masse du quadrant  
        sinon  
            critère ← appliquer le critère de Barnes_Hut  
            si critère == true:  
                On calcule la force créée par le centre de masse du quadrant  
            sinon  
                ajouter à Queue les quadrants non vides
```

# Barnes-Hut: Implémentation

**Problème**    Masse des étoiles:  $1 M_{\odot}$   
Masse du « trou noir »:  $10^6 M_{\odot}$



# Barnes-Hut: Complexité

## Création de l'arbre

Pour chacune des  $N$  étoiles, en supposant l'arbre quasi complet, on doit traverser en moyenne  $\log_2(N)$  « étages » de l'arbre.

On obtient donc une construction en  $O(N \log N)$

## Calcul des forces

Divisons l'espace autour d'une étoile comme suit.

Le nombre de cellule dans chaque anneau est  $n_{\text{sub}}^i \sim \frac{4\pi r_i^3 \theta}{\frac{4}{3}\pi r_i^3 \theta^3 / 8} = \frac{24}{\theta^2}$ .

Donc en notant  $n_0$  le nombre d'interaction au centre et  $n_{sh}$

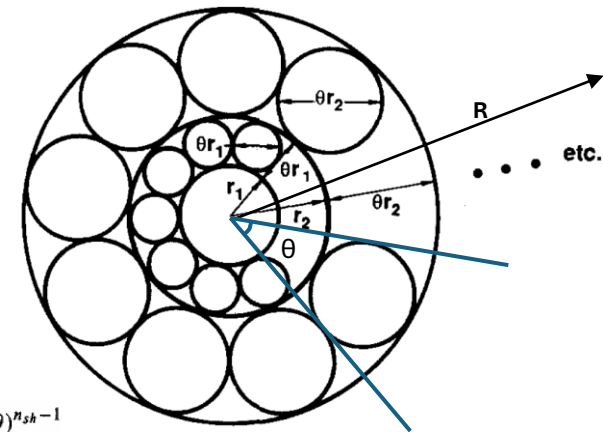
le nombre d'anneau, on obtient  $n_{\text{int}} \sim 24n_{sh}/\theta^2 + n_0$  et avec  $\frac{r_n}{r_1} = (1 + \theta)^{n_{sh}-1}$

D'où il vient  $n_{sh} = 1 + \log_{(1+\theta)} \left( \frac{1}{1+\theta} \frac{\theta}{(3N/4\pi)^{-1/3}} \right)$  par  $R = r_n(1 + \theta)$

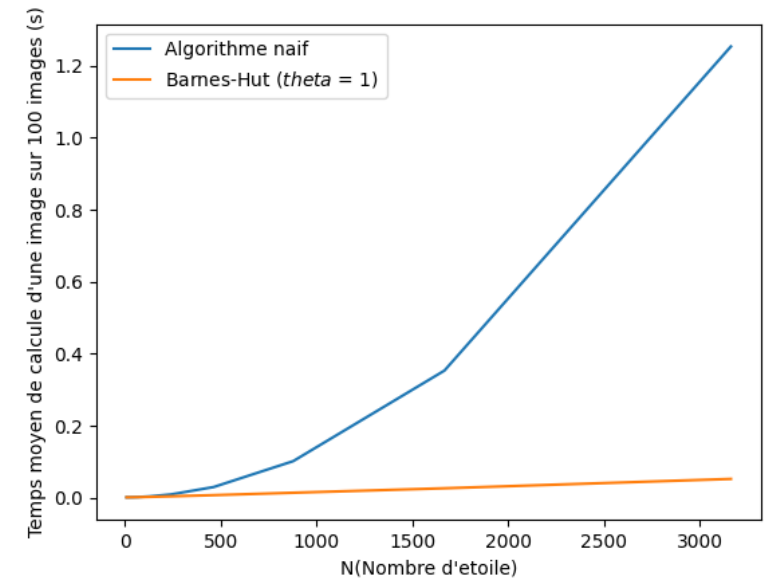
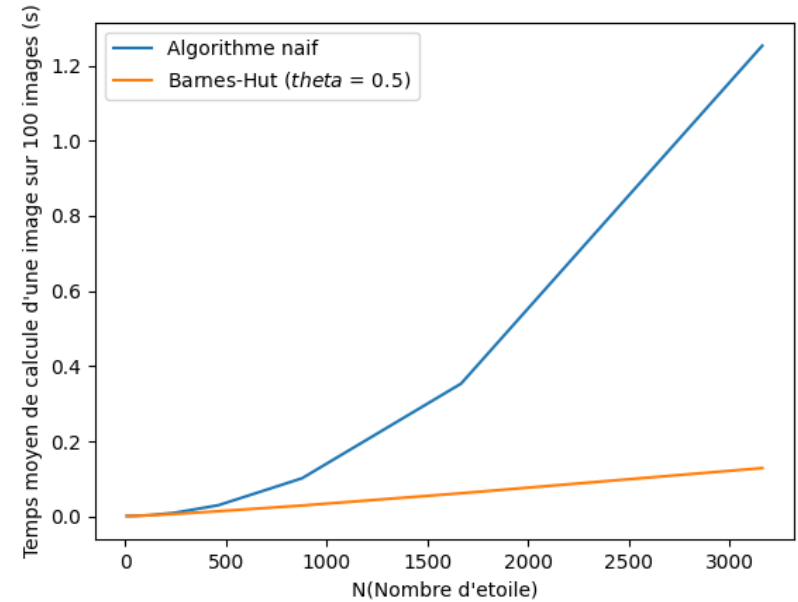
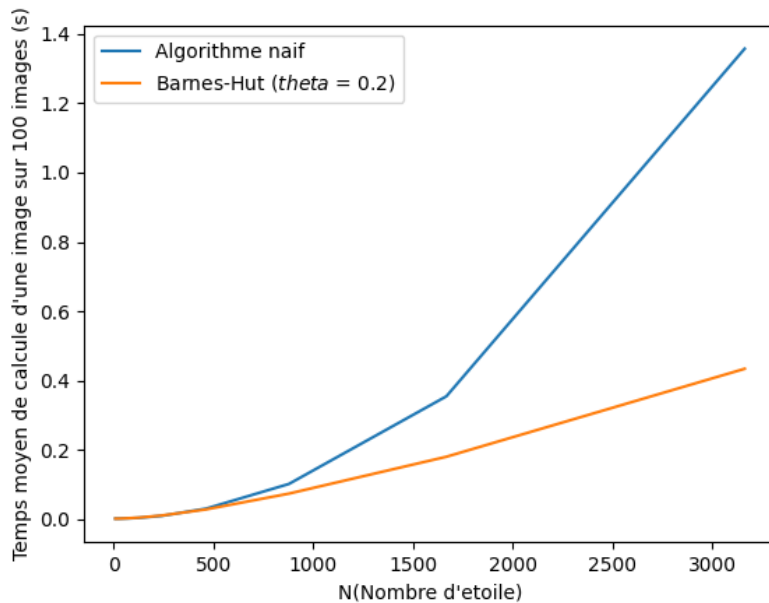
$$\text{et } n_0 = N \left( \frac{r_1}{R} \right)^3 = n \frac{4\pi}{3} r_1^3 = \frac{4\pi}{3} \frac{1}{\theta^3}$$

Ainsi  $n_{\text{int}} \sim \log N / \theta^2$

Donc pour le  $N$  étoiles, on obtient que l'algorithme est en  $O(N \log(N) / \theta^2)$

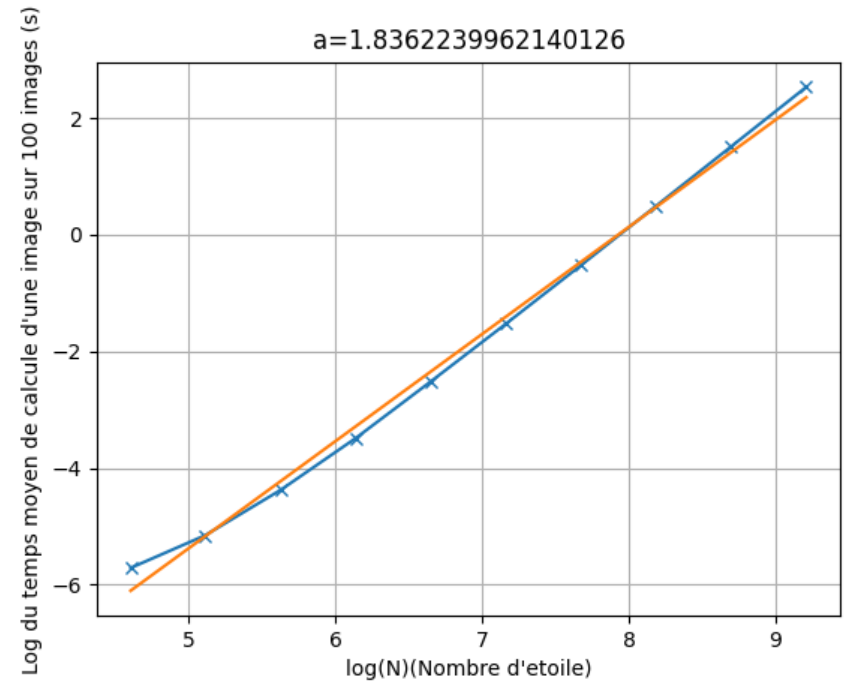
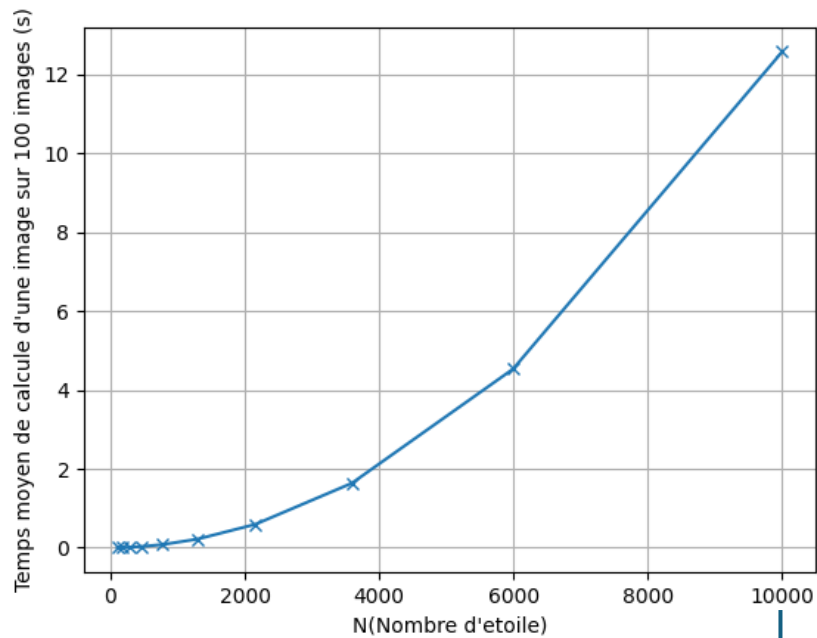


# Analyse des résultats: comparaison



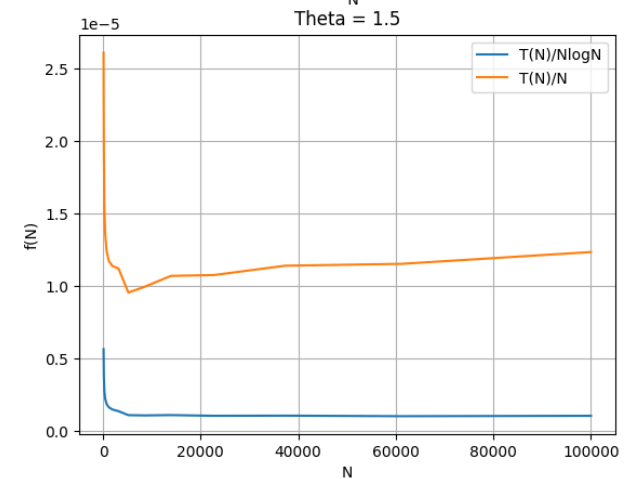
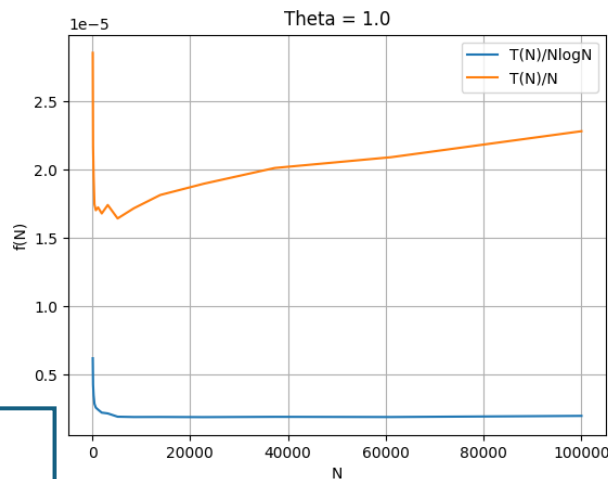
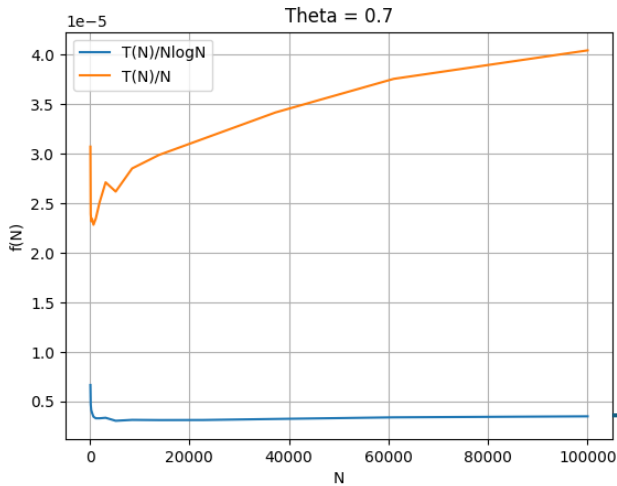
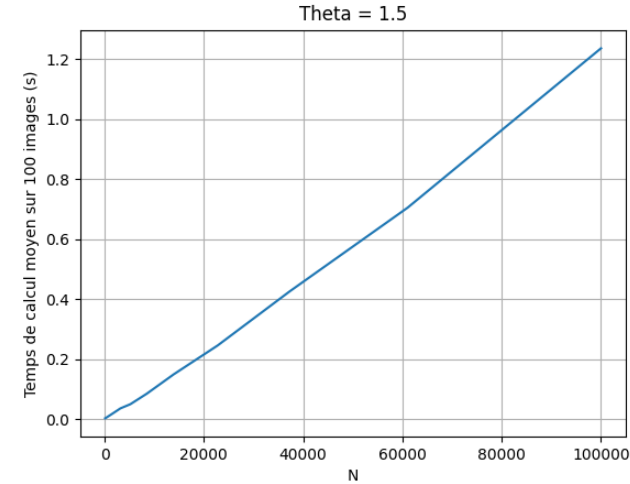
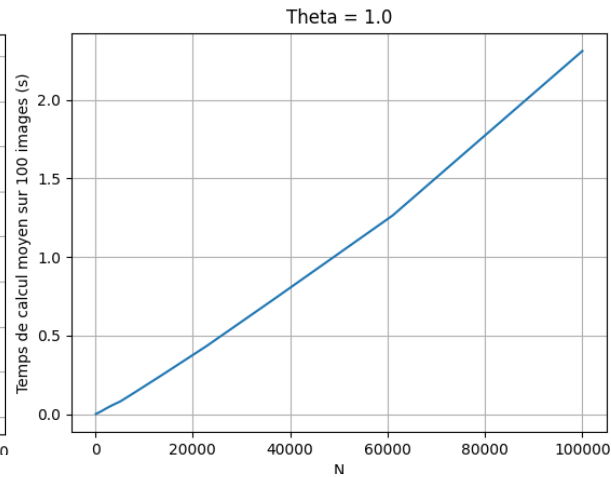
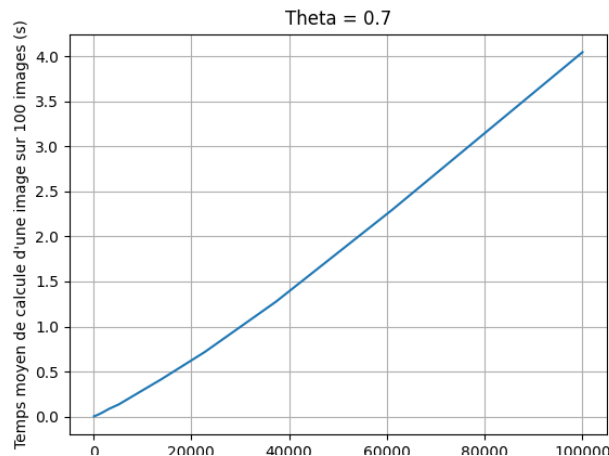


# Vérification: Naïf



Temps de calcul: 20.8 minutes

# Vérification: Barnes-Hut



Influence en  $1/\theta^2$

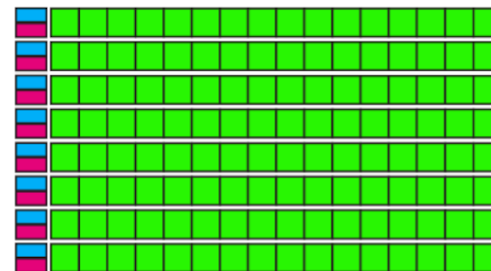
# Pistes d'amélioration

- Implémentation Quadtree peut être plus efficace
- Passage sur la carte graphique (parallélisation)

## CPU



## GPU



# Sources

## Images:

<https://www.space.com/19915-milky-way-galaxy.html>  
[https://fr.wikipedia.org/wiki/Graphe\\_complet](https://fr.wikipedia.org/wiki/Graphe_complet)  
<https://arborjs.org/docs/barnes-hut>  
<https://www.youtube.com/@physicssimulations7518>  
<https://www.lesechos.fr/>

## Documents:

- J.Dubinski: A parallel tree code: New Astronomy , 1966, volume 1 pages 133 à 147
- J.Barnes, P.Hut: A Hierarchical  $O(N \log N)$  force-calculation algorithm: Nature, 1986, Volume 327, pages 446 à 449
- S.Pfalzner, P.Gibbon: Many-Body tree methods in physics: Cambridge university press, 1996, pages 18 à 25
- T.Ventimiglia, K.Wayne: Algorithme de Barnes-Hut: <https://arborjs.org/docs/barnes-hut>
- M.Burtscher, K.Pingali: An efficient CUDA implementation of the tree-based Barnes-Hut N-body algorithm: GPU Computing Gems Emerald Edition, 2011, pages 75 à 92
- L.Hernquist: Hierarchical N-Body Methods: Computer Physics Communications 48, 1988 , pages 107 à 109

# Annexes

## Génération aléatoire des étoiles

```
std::mt19937 gen(seed);
std::normal_distribution<float> d{0, 0.7};
for(int i = 0; i < this->star_count; i++)
{
    double r = d(gen) * SCREEN_WORLD_WIDTH/2;
    double t = (PI/2) * static_cast<float>(rand()) / (static_cast<float>(RAND_MAX/3));
    this->stars[i] = Star(r*cos(t), r*sin(t), 1);
}
this->stars[0] = Star(0, 0, 4.4 * pow(10, 6));
```

# Annexes

## Fonctions d'évolution des étoiles

```
Vector2 Star::force_field(Vector2 uTov){
    double d = std::max(dist(uTov, Vector2(0, 0)), EPSILON);
    return -(K*this->mass/pow(d, 3)) * uTov;
}

void Star::apply_force(Vector2 f, double dt){
    double d = dist(dt*f, Vector2(0, 0));
    if (d < max_speed)
        this->speed += dt * f;
    else
        this->speed += dt*this->max_speed*(1/d)*f;
}

void Star::update_position(double dt){
    this->position += dt * this->speed;
}
```

# Annexes

## Création de l'arbre de Barnes-Hut

```
void Galaxy::insert_star_in_tree(double x, double y, int id, QuadTree *tree){
    if(!this->is_on_screen(id))
        return;
    if(tree->star_list.size() == 0) { // if not a leaf
        int quadrant = tree->get_quadrant(Vector2(x, y));
        if(tree->childs[quadrant] != nullptr) // if quadrant not empty, modify mass stuff and recurse into it
        {
            tree->mass_center = (tree->mass * tree->mass_center + this->stars[id].mass*this->stars[id].position)/(tree->mass + this->stars[id].mass);
            tree->mass += this->stars[id].mass;
            insert_star_in_tree(x, y, id, tree->childs[quadrant]);
        }
        else{ // if quadrant empty, then create leaf
            tree->childs[quadrant] = new QuadTree {
                {nullptr, nullptr, nullptr, nullptr},
                std::vector<int>{id},
                Vector2(x, y),
                tree->quad_center + SCREEN_WORLD_WIDTH*(pow(2, -tree->depth - 1))*Vector2(pow(-1,1-(quadrant%2)),pow(-1,(quadrant/2))),
                tree->mass,
                tree->depth + 1
            };
        }
    }
    else {
        if(tree->star_list.size() > 1 || SCREEN_WORLD_WIDTH*pow(2, -tree->depth+1) < EPSILON) { // if already multiple stars in the cell or too close to a star
            tree->mass_center = (tree->mass * tree->mass_center + this->stars[id].mass * this->stars[id].position)/(tree->mass + this->stars[id].mass);
            tree->mass += this->stars[id].mass;
            tree->star_list.push_back(id);
        }
        else{
            int last_id = tree->star_list[0];
            Vector2 last_pos = this->stars[last_id].position;
            tree->star_list = std::vector<int>();
            tree->mass_center = (tree->mass*tree->mass_center + this->stars[id].mass*this->stars[id].position)/(tree->mass + this->stars[id].mass);
            tree->mass += this->stars[id].mass;
            insert_star_in_tree(x, y, id, tree);
            insert_star_in_tree(last_pos.x, last_pos.y, last_id, tree);
        }
    }
}
```

# Annexes

## Calcul des forces selon Barnes-Hut

```
void Galaxy::Barnes_hut(int star_id){
    this->BH_queue = std::queue<QuadTree*>();
    for(int i = 0; i < 4; i++){
        if (this->BH_tree->childs[i] != nullptr)
            BH_queue.push(this->BH_tree->childs[i]);
    }

    for(; !BH_queue.empty(); BH_queue.pop())
    {
        QuadTree* current = BH_queue.front();
        if(current->star_list.size() == 1){
            Vector2 F = this->stars[current->star_list[0]].force_field(this->stars[star_id].position - this->stars[current->star_list[0]].position);
            this->stars[star_id].apply_force(F, this->time_step);
        }
        // If the star is in the quadrant's star list, don't approximate
        else if (is_in_vector(current->star_list, star_id)){
            for(int i = 0; i < current->star_list.size(); i++){
                if(current->star_list[i] == star_id)
                    continue;
                Vector2 F = this->stars[current->star_list[i]].force_field(this->stars[star_id].position - this->stars[current->star_list[i]].position);
                this->stars[star_id].apply_force(F, this->time_step);
            }
        }
        // If more than one star are in the quadrant, approximate
        else if(current->star_list.size() > 1){
            Vector2 F = Star(current->mass_center.x, current->mass_center.y, current->mass).force_field(this->stars[star_id].position - current->mass_center);
            this->stars[star_id].apply_force(F, this->time_step);
        }
        // If its not a leaf
        else{
            // Apply Barnes-Hut test
            double d = dist(this->stars[star_id].position, current->mass_center);
            double s = SCREEN_WORLD_WIDTH*pow(2, -current->depth+1);

            if (s/d < this->BH_theta){ // If sufficiently far away
                Vector2 F = Star(current->mass_center.x, current->mass_center.y, current->mass).force_field(this->stars[star_id].position - current->mass_center);
                this->stars[star_id].apply_force(F, this->time_step);
            }
            else{ // If too close
                for(int i = 0; i < 4; i++){
                    if (current->childs[i] != nullptr)
                        BH_queue.push(current->childs[i]);
                }
            }
        }
    }
}
```



# Annexes

## Fonctions de simulations des graphes (Python)

```
def gen_BH_N():
    N_list = np.logspace(2, 5, 15)
    data_list = []
    # Generate data
    for N in N_list:
        print(f"{N=}")
        t_avg = []
        seed = randint(0, 100000)
        execute(f"Simulation BH {N} 1.0 {seed} N_record.csv")
        with open("N_record.csv", "r+") as N_record:
            N_record_list = np.array(list(csv.reader(N_record)))[:,1]
            data_list.append(average(to_float_list(N_record_list)))

    with open(f"record_t.csv", "w+") as output:
        for i in range(len(N_list)):
            output.write(f"{N_list[i]}, {data_list[i]}\n")
```

```
def display_BH_N():
    t_list = []
    t_avg_list = []
    with open(f"record_t.csv", "r+") as file:
        rec = list(csv.reader(file))
        for c in rec:
            t_list.append(float(c[0]))
            t_avg_list.append(float(c[1]))
    plt.xlabel("N")
    plt.plot(t_list, t_avg_list, label="T(N)/NlogN")
    plt.ylabel(f"Temps de calcul moyen sur 100 images (s)")
    plt.title("Theta = 1.0")
    plt.grid(True)
    plt.show()
```